希望更丰富的展现？使用 Markdown、LaTeX

希望更丰富的展现？阅读 Markdown 使用帮助、KaTeX 文档。

（图源自信息学竞赛网站 www.luogu.com.cn 讨论区发布栏下侧）

注：KaTeX 与 LaTeX 在展现数学公式上几乎没有区别。具体区别见关于 LaTeX “更多”部分的介绍。

0. 引入

“你看这个 deepseek 怎么回答不了我的数学问题啊”
“你问的什么问题”
“@#&￥%P#￥……”
（使用 LaTeX 描述原问题中的公式并重新向 ds 提问）
得到正确回答。

相比于手写，使用正常的方法在电脑上输入文字，在排版和输入数学公式方面显然会遇到不少困难，不能满足我们的需要。

使用 Word，操作复杂并且对数学公式的支持不是很好，输入上下标都需要用鼠标点半天。学校的数分课本看上去是用 LaTeX 排版的。但是线代课本疑似 Word 排版，相信大家可以感受到很大的差别（实际上现在正经一点的数学课本都是 LaTeX 排版，线代课本好好反思！

了解基本的 Markdown 与 LaTeX 语法，可以帮助我们简单地解决这个问题。

如果你尝试使用过知乎或者bilibili的发表文章，会看到类似于这样的工具栏：

里面就已经集成了基本的 Markdown 与 KaTeX，可以在不知道语法的前提下完成输入。但是这样效率不是很高，并且可能导致困惑。

为了完成下文的操作，你至少需要一个支持 Markdown、LaTeX 公式的编辑器，市面上有很多，可以自行挑选。比如我是用 obsidian 写这篇文章的。直接用知乎、bilibili 的编辑器体验也不是不行。

如果要严肃地使用 LaTeX，需要更专业化的软件。我的电脑上装的是 texstudio。

每个人的偏好可能不同，仅供参考。

1. Markdown 介绍

Markdown（有时简称md，这是因为文件的后缀名是.md）支持的内容相对比较简单，是一种在原有文档上的简单扩展。好处是，它的语法极其简单，容易上手掌握。Markdown 与 HTML 兼容，不过一般来说我们不需要在文档中使用 HTML。

关于用法，查阅 Markdown 官方教程即可，里面的讲解非常系统且直观。

2. LaTeX

1. 数学公式

首先推荐一个在线编辑器，如果你不会一些操作那么可以使用它直观得到公式。

一些符号列表可以参考 这个网页；KaTeX 官方表

一般来说，键入数学公式首选 LaTeX，这是目前世界上最通用的做法。近年也有其他的方案比如typst，但是应用不广。

LaTeX 的语法相对复杂多样一些。初学的问题会不少，因此需要介绍一下。

使用 `$a+b=c$` 得到行内（行间）数学公式。效果如下：

这是一个测试 $a+b=c$ 喵呜喵呜喵

推荐在两个 `$` 号外侧添加空格，防止与文字打架。比如不加空格效果就是这样：

这是一个测试$a+b=c$喵呜喵呜喵

如果使用 `$$a+b=c$$`，可以得到行间公式。

这是一个测试

$$a+b=c$$

喵呜喵呜喵

注意一些编辑器中如果两侧的 `$` 周围有字符是不会正确显示的。

现在进入正题，到底应该如何输入复杂的数学公式呢？我们这里以一个基础（？）的例子进行讲解：

$$\begin{aligned}
ax^2+bx+c=0\ (a\neq0)\\
x_{1,2}=\frac{-b\pm\sqrt{b^2-4ac}}{2a}
\end{aligned}$$

上面这一段的源码如下：

```
$$\begin{aligned}
ax^2+bx+c=0\ (a\neq0)\\
```

```
x_{1,2}=\dfrac{-b\pm\sqrt{b^2-4ac}}{2a}
\end{aligned}$$
```

我们分别进行解释：

- `\begin{aligned}` 与 `\end{aligned}` 创建了一个可以输入多行公式的环境。否则我们输入的所有公式都只会在一行上展现。
- `ax^2`， `x_{1,2}`：上下标分别使用 `^` 与 `_` 添加。如果其后只跟一个字符，无需使用大括号；否则需要使用大括号括起来。一个经典的错误是试图输入 `10^10`，结果得到了 10^10。
- `\ `：注意数学公式中源码中所有的空格都不会显示。如果你想要在最终的公式中插入空格，需要使用反斜杠加上空格。另外有 `\quad` 等命令进行更多的间距调整操作。
- `\neq`， `\pm`， `\sqrt`：分别插入不等号，正负号，平方根。这些指令的命名有迹可循：否定的符号前面有 n 作为前缀，neq = not equal； `\pm` 这个缩的比较抽象，但是 pm = plus minus，同理输入 `\mp` 得到 $\mp$；平方根是常见英语缩写 sqrt = square root。（所以学好英语很重要）
- `\\`：进行换行。
- `\dfrac{}{}`：前面的括号里面是分子，后面的括号里面是分母。前缀 d 表示字体大小，如果直接用 `\frac` 会得到 $\frac{b}{a}$ 这种大小的分数，不是很美观；但是在指数上就不同了，$x^{\frac{a}{b}}$ 就不如 $x^{\frac{a}{b}}$，但是感觉两个的视觉效果都不如直接 $x^{a/b}$。

虽然上面的例子相对比较复杂，但是这就已经涵盖了很多常用操作了。再举一例：

$$f_{i,j}=\begin{cases}f_{i-1,j-1}+1, & a_i=b_j\\ \max(f_{i-1,j},f_{i,j-1}), & a_i\neq b_j\end{cases}$$

源代码：

```
$$
f_{i,j}=
\begin{cases}
f_{i-1,j-1}+1,&a_i=b_j\\
\max(f_{i-1,j},f_{i,j-1}),&a_i\neq b_j
\end{cases}
$$
```

（注：这是最长公共子序列问题的转移方程）

- 这里我们使用了一个新的环境 `cases`，用来打出左侧大括号的效果。
- 值得注意的是里面使用了对齐符号 `&`，它使得两行对应位置（对齐符号右侧第一个字符开始）对齐。如果不使用对齐符号，效果如下：

$$f_{i,j}=\begin{cases}f_{i-1,j-1}+1,a_i=b_j\\ \max(f_{i-1,j},f_{i,j-1}),a_i\neq b_j\end{cases}$$

值得一提的是，在 aligned 环境中也可以使用这种对齐，一般常用于比较长的公式的连等推导。

- 注意，像 $\max$ 这种符号需要使用这种字体，不要用 $max(a,b)$ 这种。对于 $\max, \min, \sup, \inf, \sin, \cos, \tan, \det, \gcd$ 等只需要多输入反斜杠即可；如果你想对其它的操作比如 lcm 也变成这种字体，可以使用 `\text{lcm}`， `\mathrm{lcm}`， `\operatorname{lcm}` 等指令。这些指令出来的效果都一样，且最标准的是最后一项，但是实践中选择代码最短的第一项（
- 一个值得注意的例外（？）是积分中 dx 的写法，有的写成 dx，有的写成 $\mathrm{d}x$（仅 d 为正体）。某种数学意义上后者更对一些，但是某个国家的标准要求使用前者。

见多识广：

$$\begin{aligned}
A(x)&=\sum\limits_{k=0}^{n-1}A(x_k)\dfrac{1}{n}\sum\limits_{j=0}^{n-1}\omega_n^{-jk}x^j\\
&=\sum\limits_{j=0}^{n-1}\left(\dfrac{1}{n}\sum\limits_{k=0}^{n-1}A(x_k)\omega_n^{-jk}\right)x^j
\end{aligned}$$

```
$$\begin{aligned}
A(x)&=
\sum\limits_{k=0}^{n-1}A(x_k)\dfrac{1}{n}\sum\limits_{j=0}^{n-1}\omega_n^{-jk}x^j\\
&=\sum\limits_{j=0}^{n-1}\left(\dfrac{1}{n}\sum\limits_{k=0}^{n-1}A(x_k)\omega_n^{-jk}\right)x^j
\end{aligned}$$
```

- 展示了 aligned 与 `&` 配合的用法。
- 这里的 `\limits` 在多行公式里其实不是必要的，但在单行公式里面会有区别：$\sum_{k=0}^{n-1}f(k)$，$\sum\limits_{k=0}^{n-1}f(k)$，二者的源码分别为 `$\sum_{k=0}^{n-1}f(k)$` 与 `$\sum\limits_{k=0}^{n-1}f(k)$`。
- 使用小写的希腊字母名称得到小写希腊字母，将首字母大写得到大写希腊字母。有的希腊字母有不同写法，加前缀 var 进行区分。比如 ϕ, Φ, φ 分别为 `\phi`， `\Phi`， `\varphi`。
- 使用形如 `\left(\right)` 的格式是必要的，这可以让括号与内部等高。否则会得到 $(\frac{1}{n})$ 这种东西。加上 left 和 right 之后得到 $\left(\frac{1}{n}\right)$。
- 上面的指令也适用于其他不同的括号，包括下取整 $\lfloor$ `\lfloor`；甚至左右的括号种类可以不同。比如一种对分数的神秘四舍五入写法 $\left\lfloor\dfrac{a}{b}\right\rceil$，源码为 `$\left\lfloor\dfrac{a}{b}\right\rceil$`。

$$\delta W=\sum_{\alpha=1}^{s}\left(Q_\alpha-\sum_{i=1}^{n}m_i\ddot{\mathbf{r}}_i\frac{\partial\mathbf{r}_i}{\partial q_\alpha}\right)\delta q_\alpha$$

```
$$
\delta W=
```

```
\sum\limits_{\alpha=1}^s
\left(
Q_{\alpha}-
\sum\limits_{i=1}^n
m_i\ddot{\textbf{r}}_i\dfrac{\partial\textbf{r}_i}{\partial q_\alpha}
\right)
\delta q_\alpha
$$
```

整体没有什么特别的，值得注意的是粗体 `\textbf` 以及上面加两个点 `\ddot`。

大家喜闻乐见的数分：

$$\forall\varepsilon>0,\exists N\in\mathbb{N},\forall n>N:|a_n-a|<\varepsilon \iff \lim_{n\to\infty}a_n=a$$

```
$$
\forall\varepsilon>0,
\exists N\in\mathbb{N},
\forall n>N:|a_n-a|<\varepsilon
\iff
\lim_{n\to\infty}a_n=a
$$
```

- 规范：请使用 `\mathbb` 调整表示数集的字母的字体。
- 关于箭头：这里偷懒写了 `\iff`，也就是 if and only if 的缩写。使用 `\Longleftrightarrow` 得到 $\Longleftrightarrow$，效果相差无几（可能长度有些许不同），大概精神是首字母大小写决定箭头中间是两道线还是一道线，是否加 `long` 决定箭头长短，`left` 和 `right` 调整箭头方向。

还有矩阵：

$$\begin{pmatrix}
1&1&1&\cdots&1\\
1&\omega_n^1&\omega_n^2&\cdots&\omega_n^{n-1}\\
1&\omega_n^2&\omega_n^4&\cdots&\omega_n^{2(n-1)}\\
\vdots&\vdots&\vdots&\ddots&\vdots\\
1&\omega_n^{n-1}&\omega_n^{2(n-1)}&\cdots&\omega_n^{(n-1)(n-1)}
\end{pmatrix}
\begin{pmatrix}
a_0\\a_1\\a_2\\ \vdots\\a_{n-1}
\end{pmatrix}=
\begin{pmatrix}
A(x_0)\\A(x_1)\\A(x_2)\\ \vdots\\A(x_{n-1})
\end{pmatrix}$$

```
$$\begin{pmatrix}
1&1&1&\cdots&1\\
1&\omega_n^1&\omega_n^2&\cdots&\omega_n^{n-1}\\
1&\omega_n^2&\omega_n^4&\cdots&\omega_n^{2(n-1)}\\
\vdots&\vdots&\vdots&\ddots&\vdots\\
1&\omega_n^{n-1}&\omega_n^{2(n-1)}&\cdots&\omega_n^{(n-1)(n-1)}
\end{pmatrix}
\begin{pmatrix}
a_0\\a_1\\a_2\\ \vdots\\a_{n-1}
\end{pmatrix}
```

```
=
\begin{pmatrix}
A(x_0)\\A(x_1)\\A(x_2)\\ \vdots\\A(x_{n-1})
\end{pmatrix}$$
```

大号矩阵一般不是人力能打出的，建议启动在线编辑器。实际上上面的所有公式都是我完全手打的，除了这一个。

值得注意的是省略号的打法，应当使用 `\cdots` 表示横向省略，`\vdots` 表示纵向省略，`\ddots` 表示左上—右下省略。一般的编辑器不支持右上到左下的省略。

以上知识基本上能应付绝大多数的数学、物理方程，最多只是要查符号表。

一个比较难绷的关于符号的视频

- 什么你说化学方程式怎么打?
  需要 mhchem 包，具体操作查阅文档，我不是很会。‘
- 怎么画图?
  有一个 tikz 包的东西可以画图，但是上手困难。建议使用平板和触控笔（
- 你说的很对，但是感觉不如在平板上手写。
  速度上，熟练背诵指令之后（我本人可能已经用这一套东西六年以上了）结合一些预设的快捷键可以做到很快，但超越手写的速度还是不太可能。但是如果和同学讨论问题，很多时候，聊天直接敲代码人肉编译的速度会比写在纸上然后拍照快（
  美观程度上，因人而异，我在平板上写东西因为滑，写快了不太能看。但是电脑排版一般情况下显然比手写要好不少。
  习惯上，我个人高强度使用电脑，当然这也与我信息竞赛的出身有关。我们学信竞的不少人都是没有纸质笔记本的（？），很多时候知识点都扔到网上的 blog 里面了。
  当众展示的时候，显然使用 LaTeX 会正式一些。不过这就涉及到一些其他的方面，参见下一小节。

2. 更多

上面提到的 KaTeX 就是一种特化的 LaTeX，是专门用来在网页上显示数学公式的。但是完整的 LaTeX 的功能不止于此，可以用来写论文，写笔记，做 ppt（?）等。

当然为了完成这些操作需要比较专业化的软件，不是只能简单用 Markdown 和 LaTeX 写笔记这样的笔记类 app。相应地在这些专业化的软件中原来 Markdown 的语法也不再受支持，需要学习 LaTeX 中对应的语法。

论文、笔记等需要一个模板。基本上就不要指望着自己能整出来很好看的模板了，直接薅别人的（开源的）就行！方便实惠，不必费尽心力排版。

这里展示一个比较朴素的笔记模板。

目录

第一章　整除理论 **1**

1.1　整除、带余除法、素数 1

1.1.1　整除 1

1.1.2　带余除法 1

1.1.3　素数与合数 2

1.1.4　算术基本定理 4

3.3.2　欧拉定理

在进行下面定理的推导之前, 我们先有一个简单的引理:

引理 3.3.3

若 r 和 r' 都通过模 m 的既约剩余系, 有 $\prod_{j=1}^{\varphi(m)} r_j \equiv \prod_{j=1}^{\varphi(m)} r'_j \pmod{m}$

实际上根据既约剩余系的性质, r' 中的数一定可表为 $r'_j = r_{\sigma(j)} + km$ 的形式. 累乘模 m 即得结论.

定理 3.3.4: 欧拉定理、费马小定理

设 $(a, m) = 1$, 则 $a^{\varphi(m)} \equiv 1 \pmod{m}$ (欧拉定理).

特别地, $a^{p-1} \equiv 1 \pmod{p}$ (费马小定理).

证明. 根据引理和 3.2.4 的性质 3, 考虑一组通过模 m 的既约剩余系 r 以及与 m 互素的 a, 我们有

$$\prod_{j=1}^{\varphi(m)} r_j \equiv \prod_{j=1}^{\varphi(m)} (ar_j) \equiv a^{\varphi(m)} \prod_{j=1}^{\varphi(m)} r_j \pmod{m}$$

上面图片有显示不清晰的地方是因为太细了，在原 pdf 上放大是完整的。

下面的图片展示了上方截图背后的源代码。注意公式是自动编号的！

```latex
\subsection{欧拉定理}

在进行下面定理的推导之前, 我们先有一个简单的引理:

\begin{lem}{}{}
若 $r$ 和 $r'$ 都通过模 $m$ 的既约剩余系, 有 $\Prod_{j=1}^{\varphi(m)} r_j\equiv \Prod_{j=1}^{\varphi(m)}r_j'\pmod m$
\end{lem}

实际上根据既约剩余系的性质, $r'$ 中的数一定可表为 $r_j'=r_{\sigma(j)}+km$ 的形式. 累乘模 $m$ 即得结论.

\begin{thm}{欧拉定理、费马小定理}{}
    设 $(a,m)=1$, 则 $a^{\varphi(m)}\equiv1\pmod m$ (欧拉定理). \\
    特别地, $a^{p-1}\equiv1\pmod p$ (费马小定理).
\end{thm}

\begin{proof}
根据引理和 \ref{thm:modseries} 的性质 3, 考虑一组通过模 $m$ 的既约剩余系 $r$ 以及与 $m$ 互素的 $a$, 我们有
$$\Prod_{j=1}^{\varphi(m)}r_j\equiv \Prod_{j=1}^{\varphi(m)}(ar_j)\equiv a^{\varphi(m)}\Prod_{j=1}^{\varphi(m)}r_j\pmod m$$

因为 $r$ 为既约剩余系, $(r_j,m)=1$, 故
$$a^{\varphi(m)}\equiv1\pmod m$$

费马小定理作为推论自然成立.
\end{proof}
```

再展示一个信息竞赛模拟赛的模板：

NOIP 模拟赛

时间： 2025 **年** 8 **月** 18 **日**

题目名称	背包	时官	节日	变易
题目类型	传统型	传统型	Special Judge	传统型
目录	`knapsack`	`clock`	`festival`	`transform`
可执行文件名	`knapsack`	`clock`	`festival`	`transform`
输入文件名	`knapsack.in`	`clock.in`	`festival.in`	`transform.in`
输出文件名	`knapsack.out`	`clock.out`	`festival.out`	`transform.out`
每个测试点时限	1.0s	1.0s	1.0s	1.0s
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	10	15	10	10
测试点是否等分	是	否	是	是
是否进行捆绑测试	否	是	否	是

提交源程序文件名

对于 C++ 语言	`knapsack.cpp`	`clock.cpp`	`festival.cpp`	`transform.cpp`

编译选项

对于 C++ 语言	`-O2 -std=c++14 -static`

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参照具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈内存空间限制与题目的内存限制一致。

【样例 2 输入】

```
5 3
1 3 2 4 3
```

【样例 2 输出】

```
3
```


【数据范围】

测试点编号	$n \le$	$k \le$	$a_i \le$
$1 \sim 3$	100	100	20
$4, 5$	5×10^3	10^6	10^5
$6, 7$	10^6	10^6	10^5
$8 \sim 10$	10^6	10^{17}	10^{12}

对于 100% 的数据，保证有 $1 \le n \le 10^6, 0 \le k \le 10^{17}, 0 \le a_i \le 10^{12}$。

使用 beamer 包甚至可以做 ppt（成品是 pdf 格式的）

注：下面这张 ppt 不是我做的。希望不要侵权。

基础数论
└整除
　└最大公约数

最小公倍数

一个显然而重要的公式是

$$\mathrm{lcm}(a,b) = \frac{ab}{\gcd(a,b)}$$

这个公式给出了 lcm 与 gcd 的关系。

需要注意的是，在实际的代码实现中，为避免中间结果的溢出，应该使用 `a/gcd(a,b)*b` 而不是 `a*b/gcd(a,b)`。

这里是因为截图所以文字位置看上去有些怪。大家体会效果就好。

3. 总结

只是介绍一套工具，希望可能有用。

在这样的短文章中覆盖 Markdown 与 LaTeX 的所有方面显然不现实。更多的知识，还是自己实践、搜索资料得到。